

**CSE4067**

# **Introduction to Blockchain Programming**

**Goshgar Can Ismayilov**

[goshgar.ismayilov@boun.edu.tr](mailto:goshgar.ismayilov@boun.edu.tr)

<https://www.linkedin.com/in/gismayilov/>

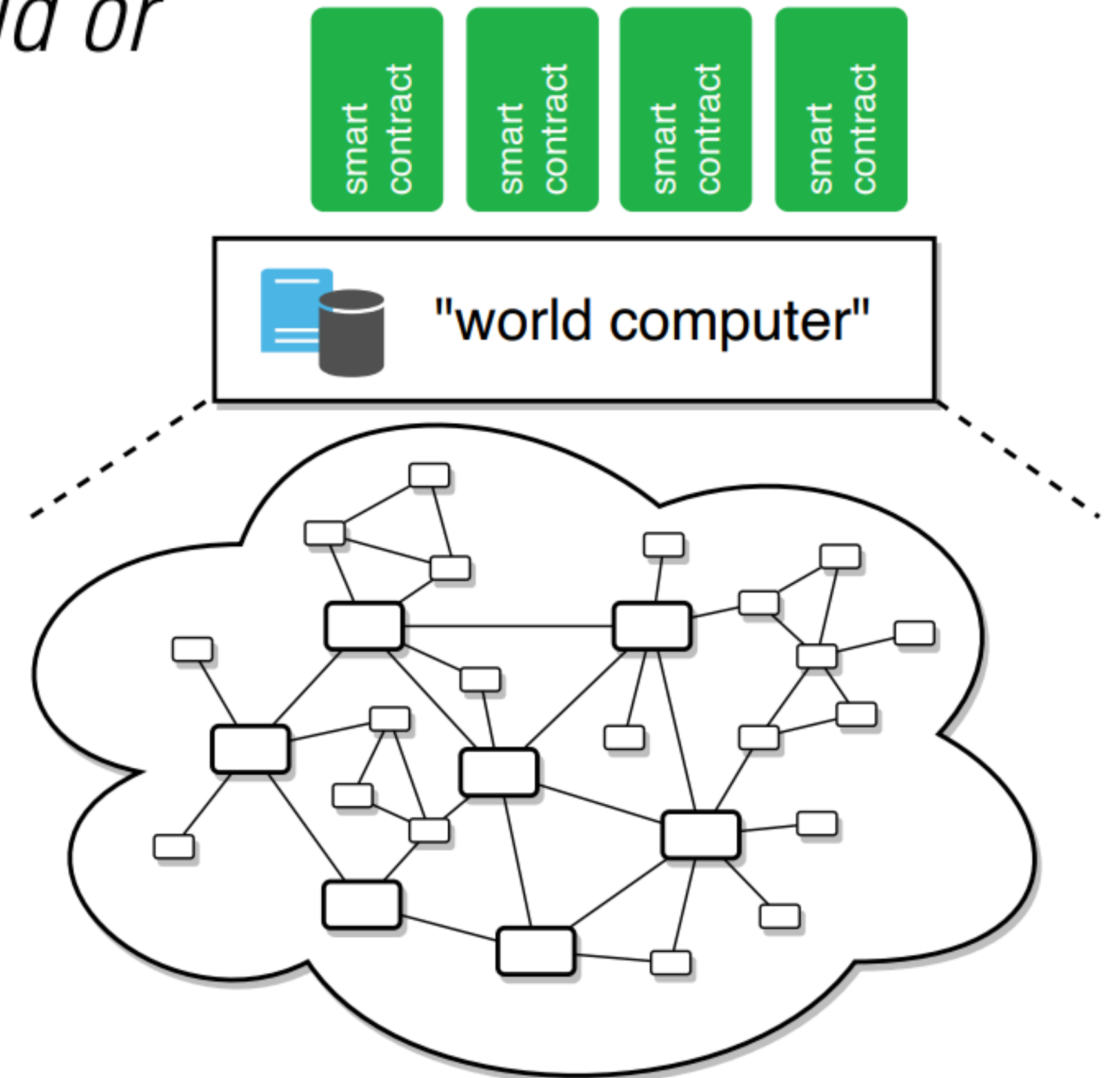
22.10.2024

# Types of Blockchains

|                             | Permissionless Blockchain | Permissioned Blockchain       | Central Database |
|-----------------------------|---------------------------|-------------------------------|------------------|
| Throughput                  | Low                       | High                          | Very High        |
| Latency                     | Slow                      | Medium                        | Fast             |
| Number of readers           | High                      | High                          | High             |
| Number of writers           | High                      | Low                           | High             |
| Number of untrusted writers | High                      | Low                           | 0                |
| Consensus mechanism         | Mainly PoW, some PoS      | BFT protocols (e.g. PBFT [5]) | None             |
| Centrally managed           | No                        | Yes                           | Yes              |

# Smart Contracts

*"Ethereum is a decentralized platform that runs smart contracts: applications that run exactly as programmed without any possibility of downtime, censorship, fraud or third-party interference."*



# Smart Contracts

## con·tract

(noun) /'käntrakt/

1. a written or spoken agreement ... that is intended to be enforceable by law.

## smart con·tract

(noun) /smärt 'käntrakt/

1. code that **facilitates**, **verifies**, or **enforces** the negotiation or execution of a digital contract.
  - a. **Trusted entity** must run this code

# Smart Contracts

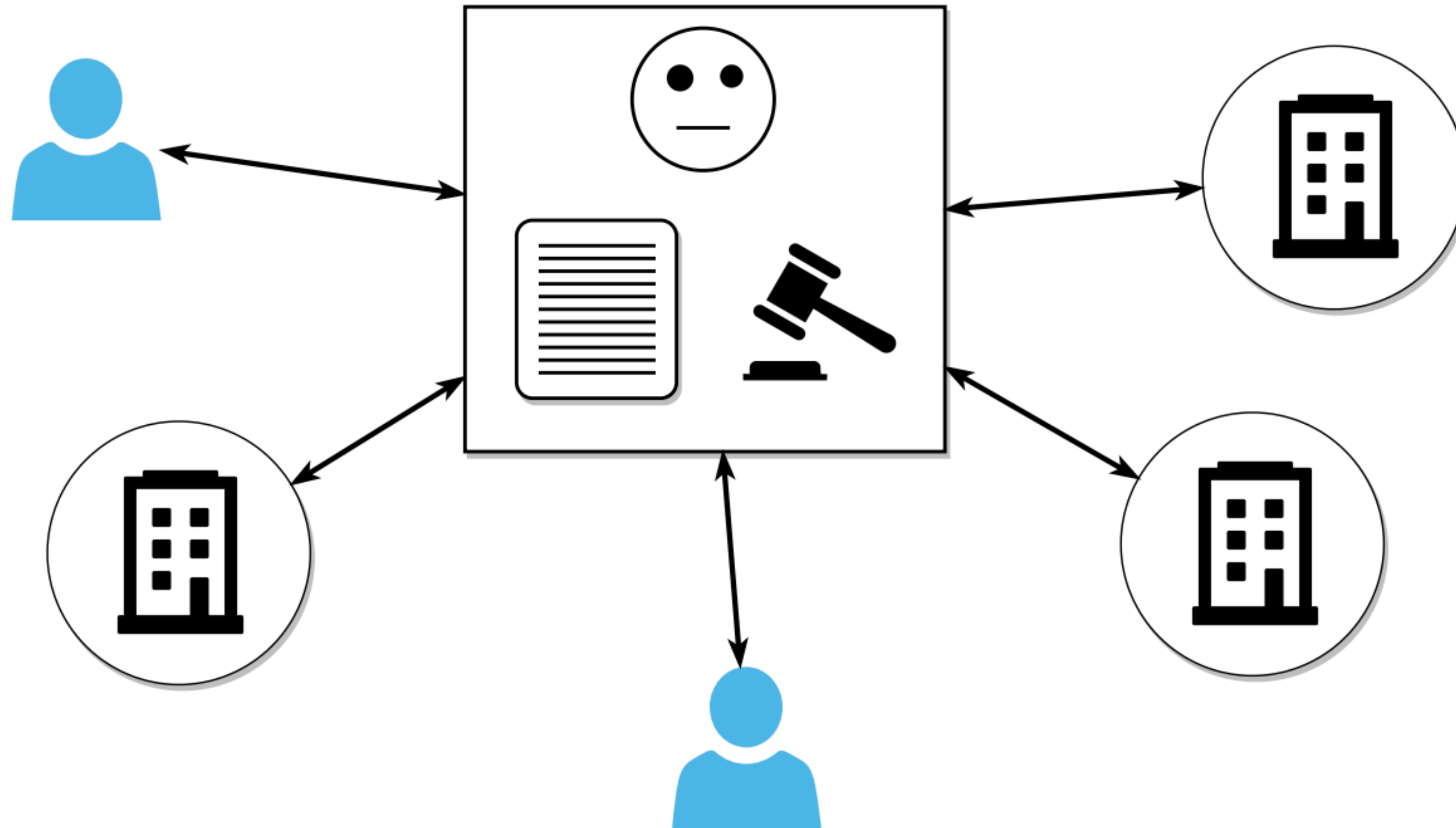
"A **smart contract** is a computer protocol intended to digitally facilitate, verify, or enforce the negotiation or performance of a contract.

Smart contracts allow the performance of credible transactions without third parties.

These transactions are trackable and irreversible."

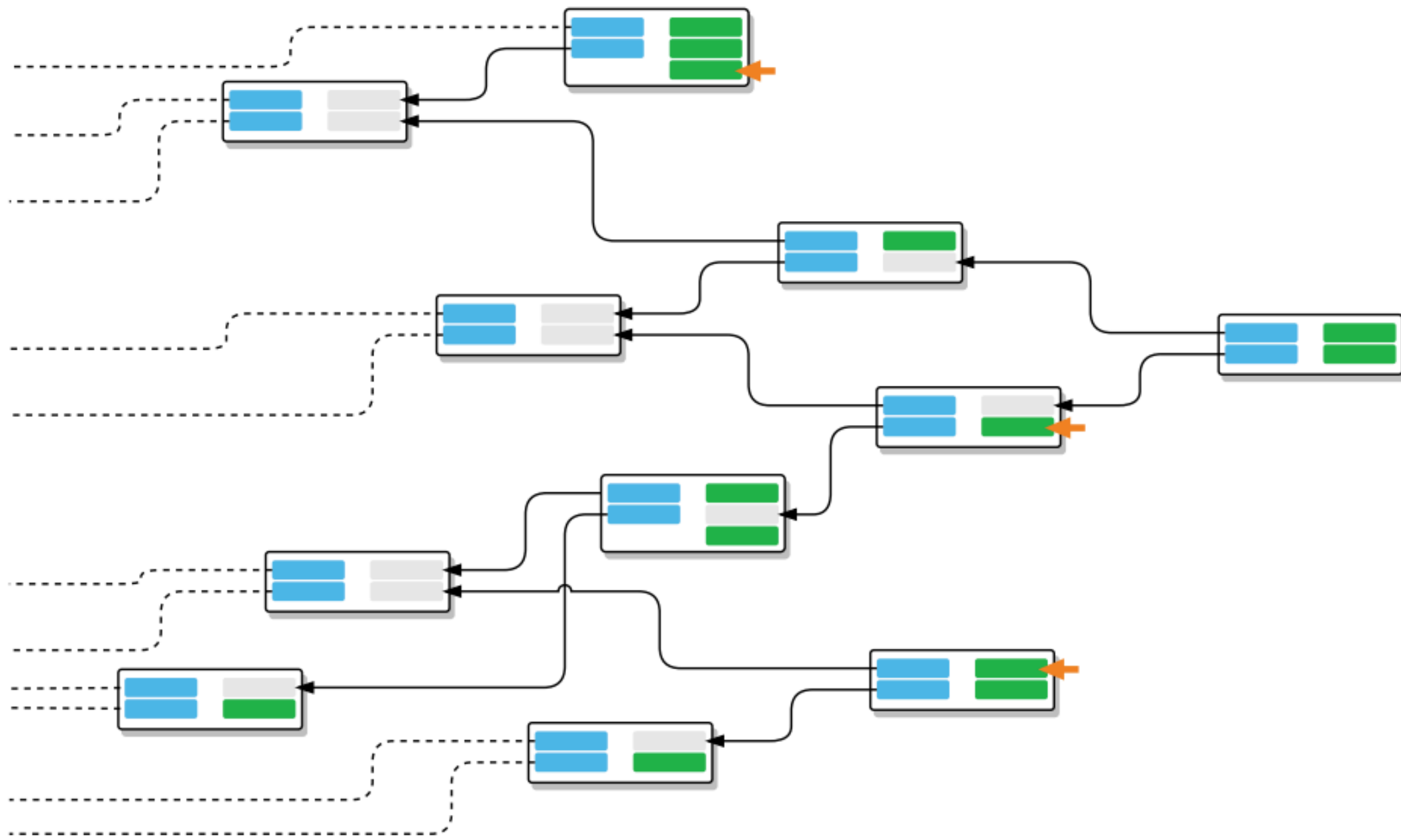
# Smart Contracts

AN INCORRUPTIBLE 3RD PARTY



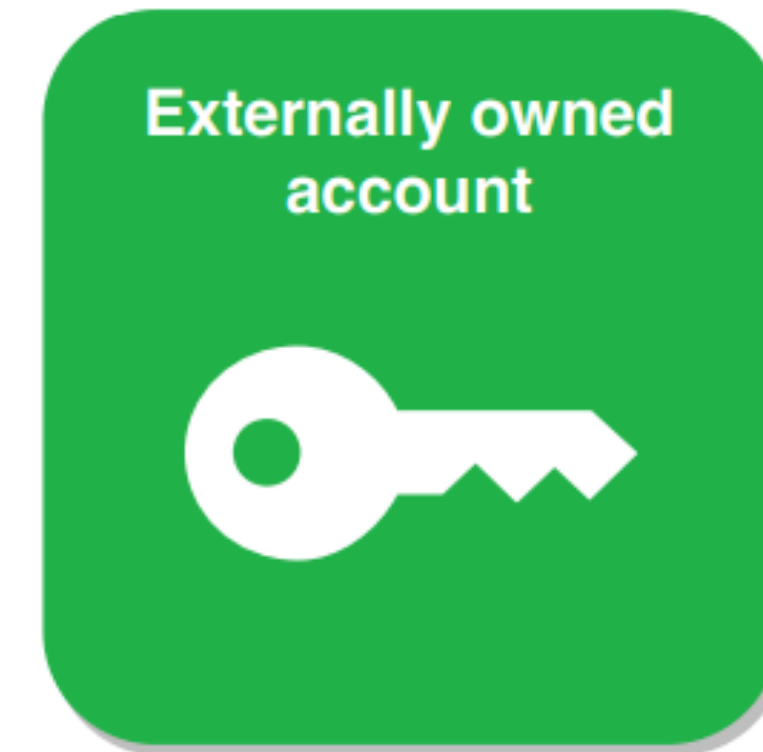
# Smart Contracts

BITCOIN USES **UTXO**



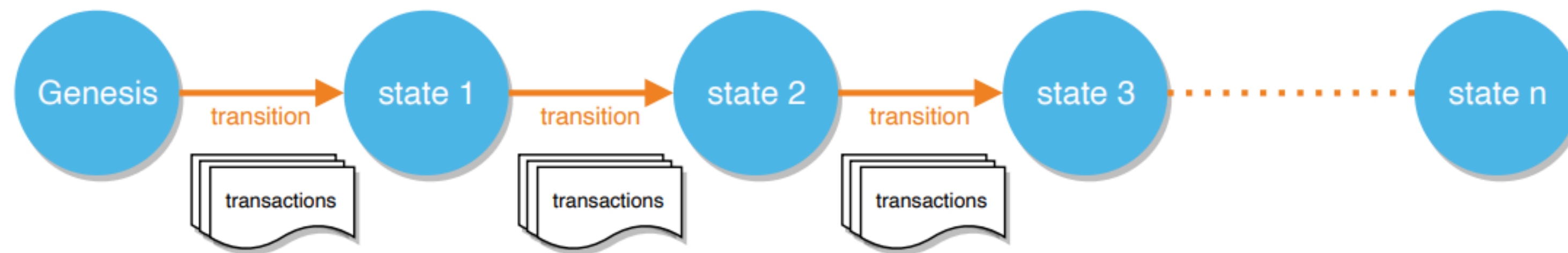
User owns private keys to a set of UTXOs

ETHEREUM USES **ACCOUNTS**



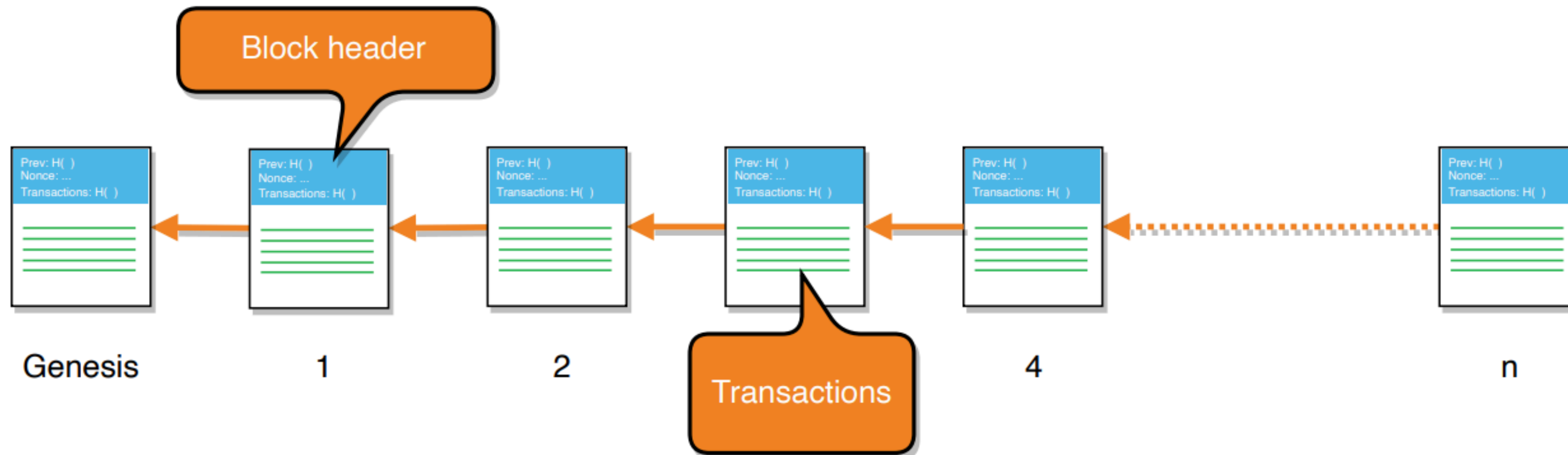
# Smart Contracts

ETHEREUM: A **DISTRIBUTED STATE MACHINE** THAT RELIES ON **TRANSACTIONS** TO MOVE BETWEEN STATES.



# Smart Contracts

NETWORK STATE IS UPDATED WITH EVERY **BLOCK**

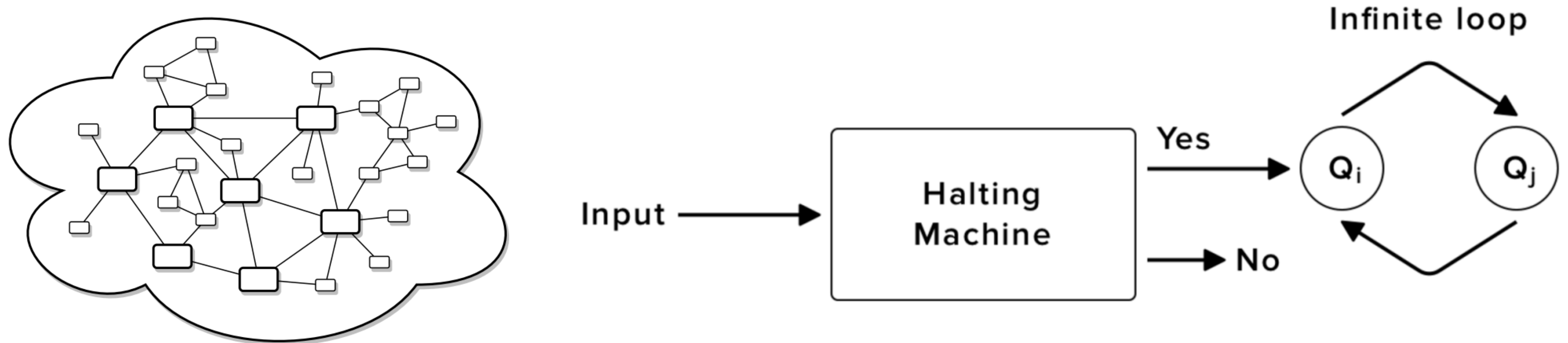


# Smart Contracts

BITCOIN SCRIPT: NOT TURING-COMPLETE

THE **ETHEREUM VIRTUAL MACHINE** (EVM) RUNS THIS  
CODE

EVERY NODE RUNS THE CODE AS PART OF TRANSACTION  
VALIDATION



# Smart Contracts

## EVM DESIGN GOALS

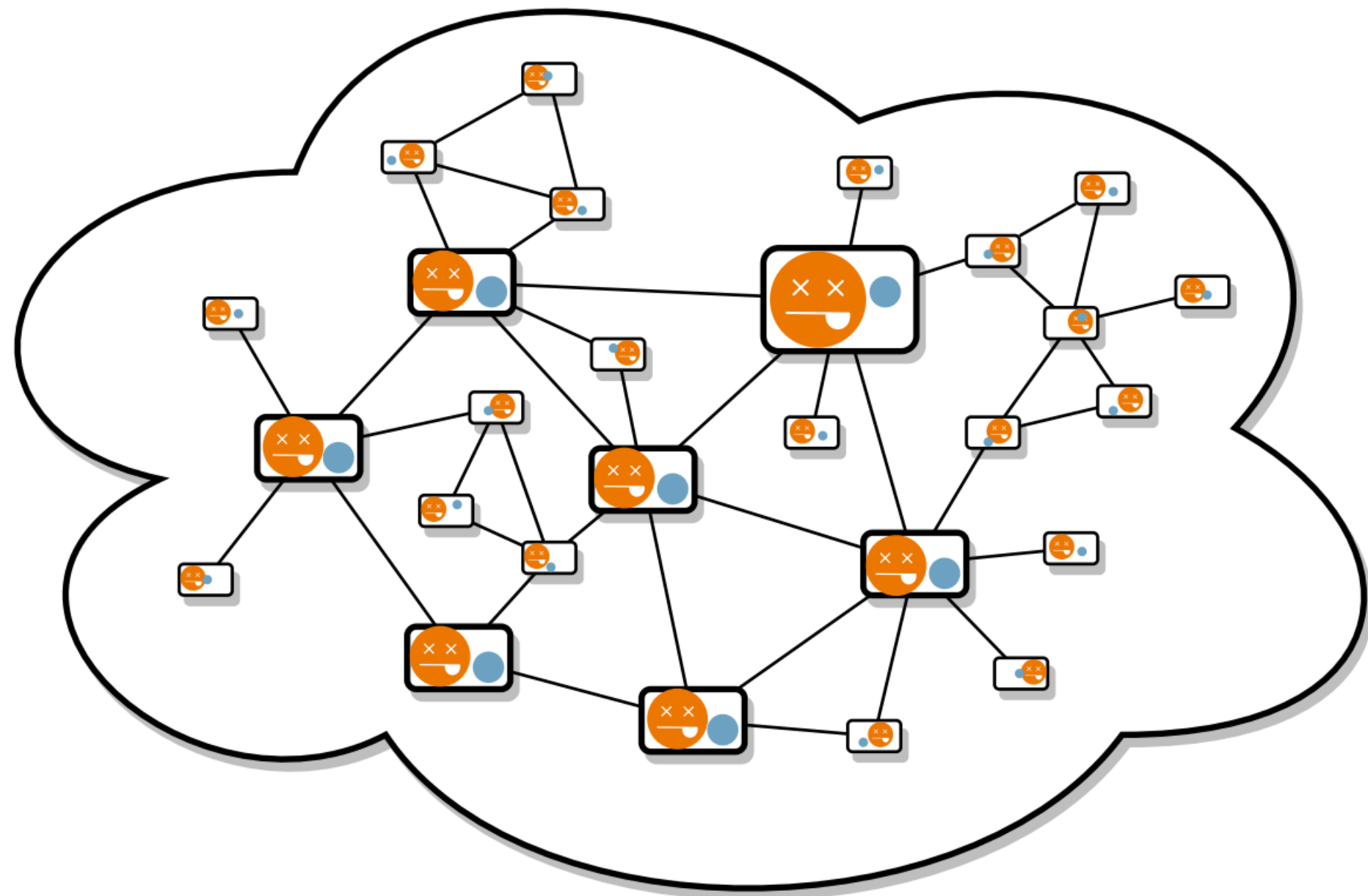
- **Simplicity:** low-level opcodes, limited in number
- **Determinism:** same input state should always yield the same output state
- **Space efficiency:** EVM code stored on blockchain, so as small as possible
- **Specialization:** operations tailor made for smart contracts
- **Security:** easy to prevent DoS attacks

THE EVM IS **TURING COMPLETE!**

# Smart Contracts

```
int c = 0;  
while (1) {  
    // do stuff  
    c += 1;  
}
```

ALL NODES WOULD GRIND TO A HALT!



ETHEREUM'S SOLUTION: **GAS**



# Smart Contracts

|                    |       |                                                                                                     |
|--------------------|-------|-----------------------------------------------------------------------------------------------------|
| $G_{zero}$         | 0     | Nothing paid for operations of the set $W_{zero}$ .                                                 |
| $G_{base}$         | 2     | Amount of gas to pay for operations of the set $W_{base}$ .                                         |
| $G_{verylow}$      | 3     | Amount of gas to pay for operations of the set $W_{verylow}$ .                                      |
| $G_{low}$          | 5     | Amount of gas to pay for operations of the set $W_{low}$ .                                          |
| $G_{mid}$          | 8     | Amount of gas to pay for operations of the set $W_{mid}$ .                                          |
| $G_{high}$         | 10    | Amount of gas to pay for operations of the set $W_{high}$ .                                         |
| $G_{extcode}$      | 700   | Amount of gas to pay for operations of the set $W_{extcode}$ .                                      |
| $G_{balance}$      | 400   | Amount of gas to pay for a BALANCE operation.                                                       |
| $G_{sload}$        | 200   | Paid for a SLOAD operation.                                                                         |
| $G_{jumpdest}$     | 1     | Paid for a JUMPDEST operation.                                                                      |
| $G_{sset}$         | 20000 | Paid for an SSTORE operation when the storage value is set to non-zero from zero.                   |
| $G_{sreset}$       | 5000  | Paid for an SSTORE operation when the storage value's zeroness remains unchanged or is set to zero. |
| $R_{sclear}$       | 15000 | Refund given (added into refund counter) when the storage value is set to zero from non-zero.       |
| $R_{selfdestruct}$ | 24000 | Refund given (added into refund counter) for self-destructing an account.                           |
| $G_{selfdestruct}$ | 5000  | Amount of gas to pay for a SELFDESTRUCT operation.                                                  |
| $G_{create}$       | 32000 | Paid for a CREATE operation.                                                                        |
| $G_{codedeposit}$  | 200   | Paid per byte for a CREATE operation to succeed in placing code into state.                         |
| $G_{call}$         | 700   | Paid for a CALL operation.                                                                          |
| $G_{callvalue}$    | 9000  | Paid for a non-zero value transfer as part of the CALL operation.                                   |
| $G_{callstipend}$  | 2300  | A stipend for the called contract subtracted from $G_{callvalue}$ for a non-zero value transfer.    |

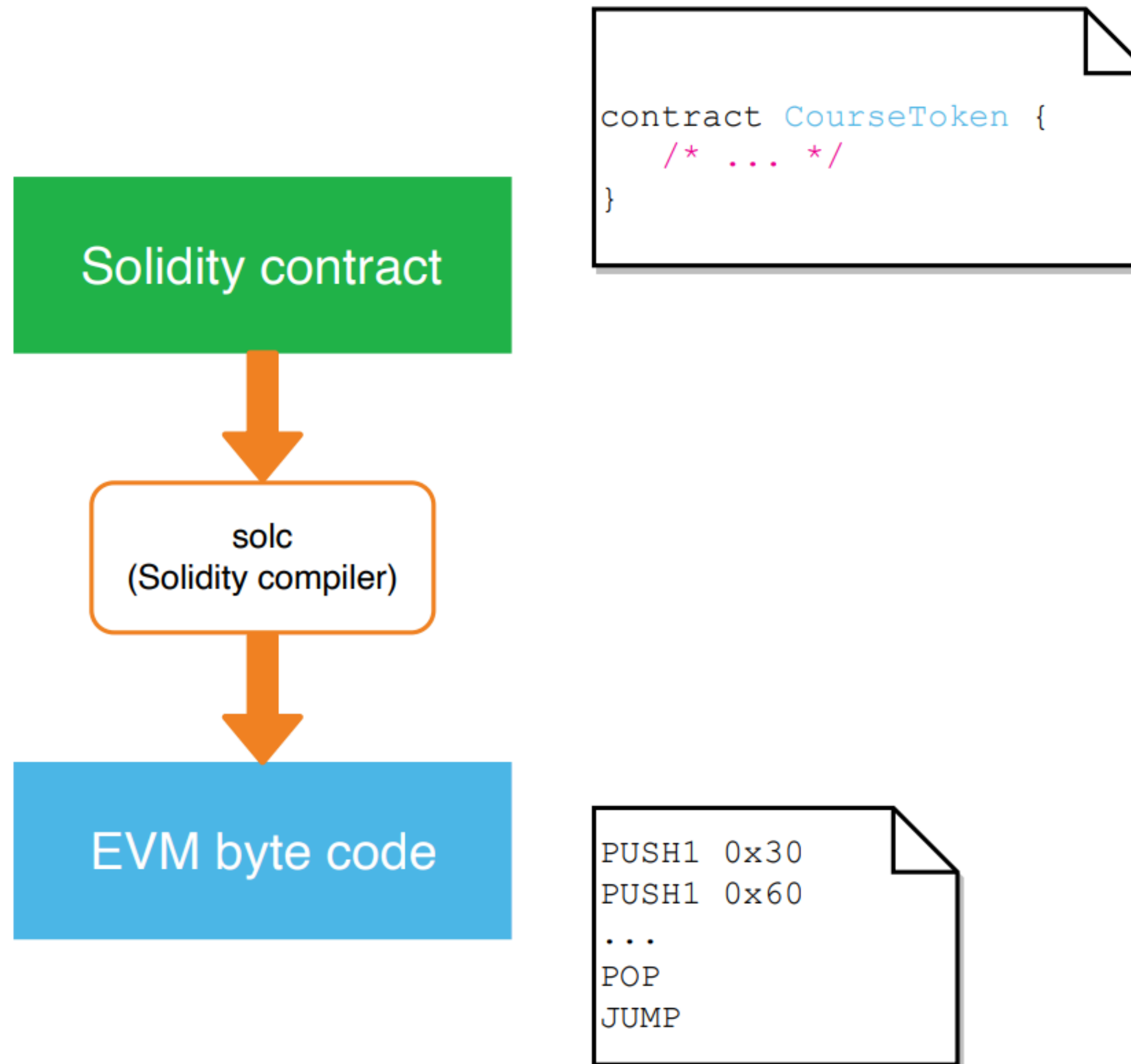
# Smart Contracts

# BUT WHO WANTS TO WRITE EVM ASSEMBLY?

```
[1] PUSH1 0x60  
[3] PUSH1 0x40  
[4] MSTORE  
[6] PUSH1 0x04  
[7] CALLDATASIZE  
[8] LT  
[11] PUSH2 0x0062  
[12] JUMPI  
[14] PUSH1 0x00  
[15] CALLDATALOAD  
[45] PUSH29 0x0100000000000000000000000000000000000000000000000000000000000000  
[46] SWAP1  
[47] DIV  
[52] PUSH4 0xffffffff  
[53] AND  
[54] DUP1
```

# Smart Contracts

## SOLIDITY: HIGH-LEVEL CONTRACT LANGUAGE



# Smart Contracts

Smart contracts

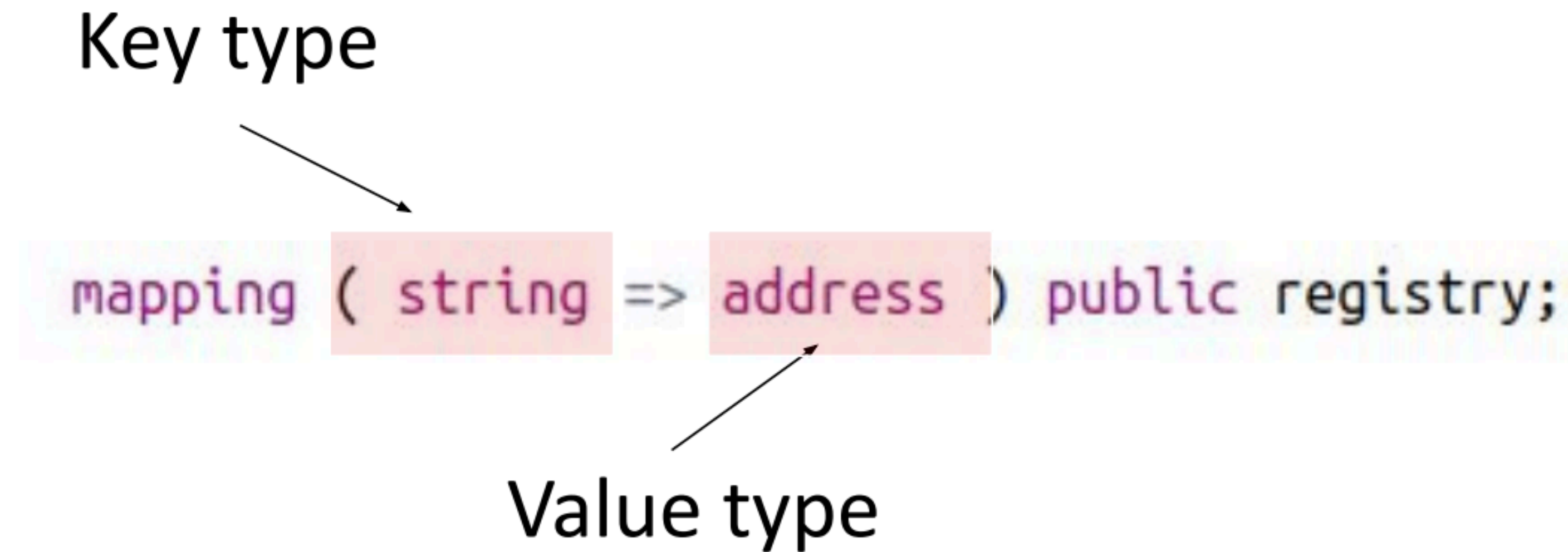


- Controlling high-value assets
- Unchangeable
- Unstoppable
- Autonomous
- Running in an open, hostile environment
- Code written by **fallible humans?**

# Smart Contracts

- *Mapping*: a key value storage / hash table
- Every key is initially mapped to zero

Key type



```
mapping ( string => address ) public registry;
```

Value type

- There is no built-in way to query the length of a mapping, or iterate over its non-zero elements. A separate variable can be used

# Smart Contracts

```
mapping(string => uint256) balances;
```

|       |     |
|-------|-----|
| Alice | 15  |
| Bob   | 15  |
| Joe   | 100 |



- every item requires at least one 256-bit word
- `balances["Andrew"]` is 0 if "Andrew" doesn't exist or if "Andrew" has 0 balance
- to delete a key, set `balances["Andrew"] = 0`
- Cannot delete an entire map!

# Smart Contracts

Statically sized array:

```
int32[10] memory fixSizeArray;  
fixSizeArray[2] = 15;  
fixSizeArray[5] = 30;
```

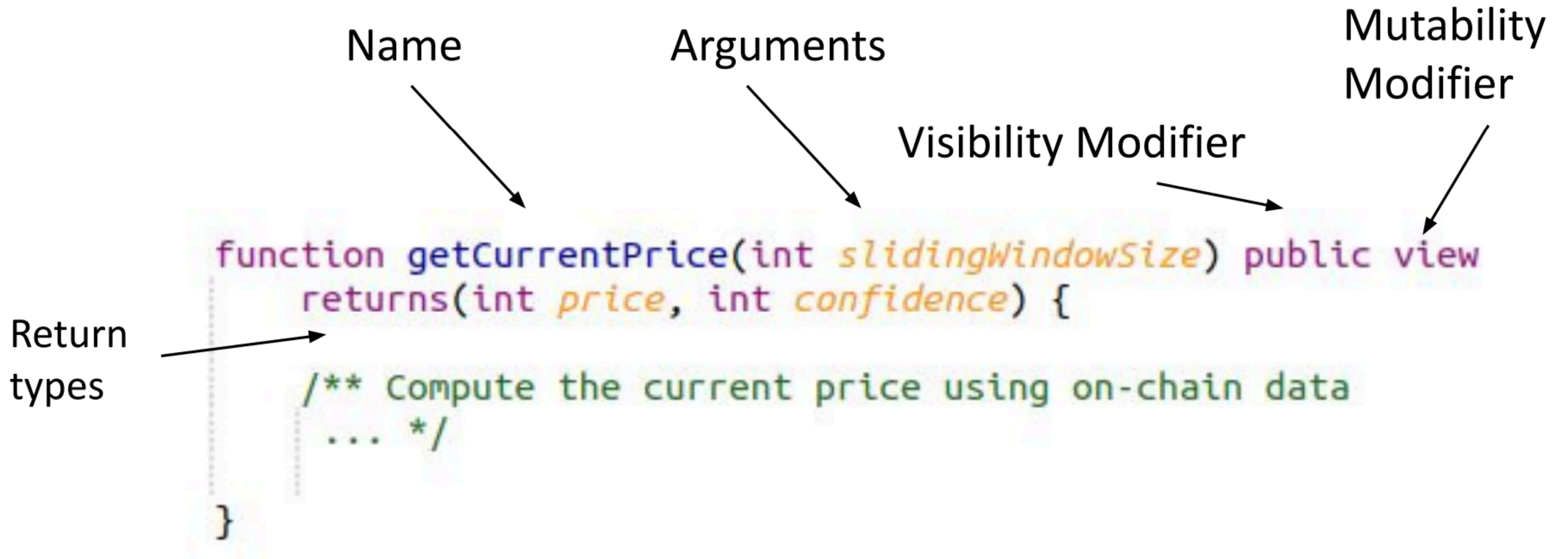
Dynamic length array:  
(more expensive,  
still can't change once created)

```
int32[] memory varSizeArray = new int32[](x);  
varSizeArray[2] = 15;  
varSizeArray[5] = 30;
```

Array in storage:  
(persists across  
transactions)

```
address[] listOfCallers;  
  
function append() public returns(uint) {  
    listOfCallers.push(msg.sender);  
    return listOfCallers.length;  
}
```

# Smart Contracts



# Smart Contracts

```
contract Namespace {  
  
    struct NameEntry {  
        address owner;  
        bytes32 value;  
    }  
  
    uint32 constant REGISTRATION_COST = 100;  
    uint32 constant UPDATE_COST = 10;  
    mapping(bytes32 => NameEntry) data;  
  
    function nameNew(bytes32 hash){  
        if (msg.value >= REGISTRATION_COST){  
            data[hash].owner = msg.sender;  
        }  
    }  
  
    function nameUpdate(bytes32 name, bytes32 newValue, address newOwner){  
        bytes32 hash = sha3(name);  
        if (data[hash].owner == msg.sender && msg.value >= UPDATE_COST){  
            data[hash].value = newValue;  
            if (newOwner != 0){  
                data[hash].owner = newOwner;  
            }  
        }  
    }  
  
    function nameLookup(bytes32 name){  
        return data[sha3(name)];  
    }  
  
}
```

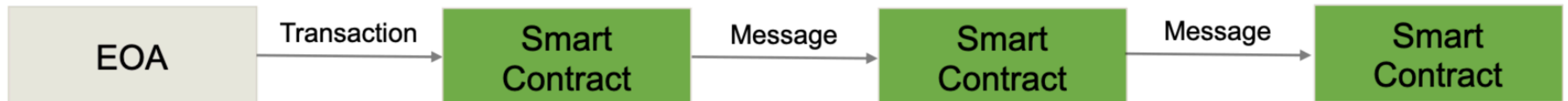
# Smart Contracts

The interface for an external contract

```
abstract contract Token {  
    function transferFrom(address from, address to, uint amount) public virtual;  
}  
  
contract Exchanger {  
    Token tokenA = Token(address(0x000 /* Hardcoded address of existing token */ ));  
    Token tokenB = Token(address(0x000 /* Hardcoded address of existing token */ ));  
  
    function swap1(address Alice, address Bob) public {  
        tokenA.transferFrom(Alice, Bob, 1);  
        tokenB.transferFrom(Bob, Alice, 1);  
    }  
}
```

Address of external  
contract instance

Method Call



# Smart Contracts

```
function acceptExactlyTwoEther() public payable returns(uint) {  
    require(msg.value >= 2.0 ether);  
  
    uint refund = msg.value - 2.0 ether;  
    payable(msg.sender).transfer(refund);  
  
    return address(this).balance;  
}
```

1.0 ether => 10000000000000000000000 wei

# Smart Contracts

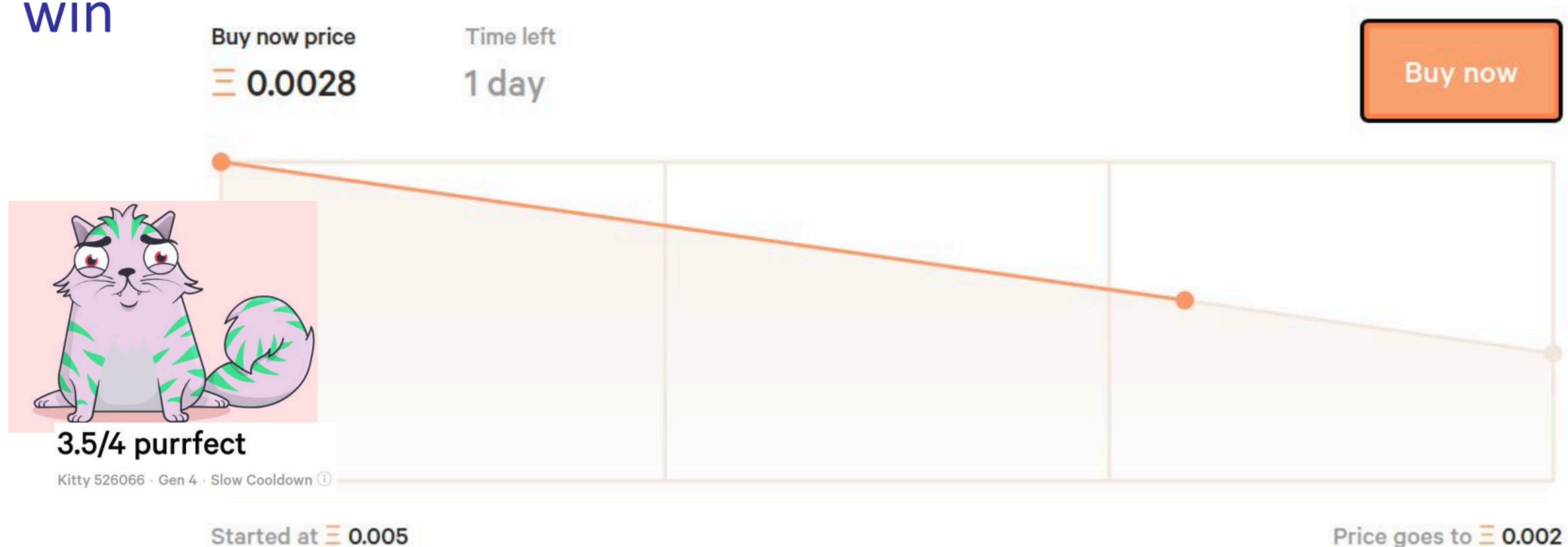
```
function placeBid(int price) public {  
    require(block.timestamp <= deadline);  
  
    /** rest of the code for placing a bid **/  
}
```

# Cryptokitties and Dutch Auction

The “Buy it Now” price is initially set at a largest value

As time goes on, the “Buy it Now” price is lowered

As soon as someone is ready to buy it, they announce their bid and win



# Smart Contracts

```
contract DutchAuction {  
    // Parameters  
    uint public initialPrice; uint public biddingPeriod;  
    uint public offerPriceDecrement; uint public startTime;  
    KittyToken public kitty; address payable public seller;  
    address payable winnerAddress;  
  
    function buyNow() public payable {  
        uint timeElapsed = block.timestamp - startTime;  
        uint currPrice = initialPrice - (timeElapsed * offerPriceDecrement);  
        uint userBid = msg.value;  
        require (winnerAddress == address(0)); // Auction hasn't ended early  
        require (timeElapsed < biddingPeriod); // Auction hasn't ended by time  
        require (userBid >= currPrice); // Bid is big enough  
  
        winnerAddress = payable(msg.sender);  
        winnerAddress.transfer(userBid - currPrice); // Refund the difference  
        seller.transfer(currPrice);  
        kitty.transferOwnership(winnerAddress);  
    }  
}
```